

Alcance de variables

Clase 11

Subrutinas (métodos, funciones, procedimientos)

- **Función:** Retorna un valor (return).
- **Método:** Asociada a la Programación Orientada a Objetos.
- **Procedimiento:** No retorna ningún valor.

```
def funcion1(param1, param2):
```

```
    # Código1
```

```
    return resultado
```

```
    # Código2
```

```
result = funcion1()
```



Parámetros con valor por defecto

```
def funcion1(a, b, c, d=4):
```

```
    print(a, b, c, d)
```

```
def funcion2(a, b, c=3, d=4):
```

```
    print(a, b, c, d)
```

```
def funcion3(a=1, b=2, c, d): # Error
```

```
    print(a, b, c, d)
```



Cantidad de parámetros indeterminado (*args)

```
def funcion(a, b, *args):  
    print(a, b, end=' ')  
    for elem in args:  
        print(elem, end=' ')
```

```
funcion(1, 2)           # 1 2
```

```
funcion(1, 2, 3, 'a', []) # 1 2 3 'a' []
```



Cantidad de parámetros indeterminado (**kwargs)

```
def funcion(a, b, **kwargs):  
    print(a, b, end=' ')  
    for key in kwargs:  
        print(key, kwargs[key], end=' ')  
    print('')
```

```
funcion(1, 2)                # 1 2
```

```
funcion(1, 2, seis=6, siete=7)  # 1 2 seis 6 siete 7
```



Anotaciones de tipo

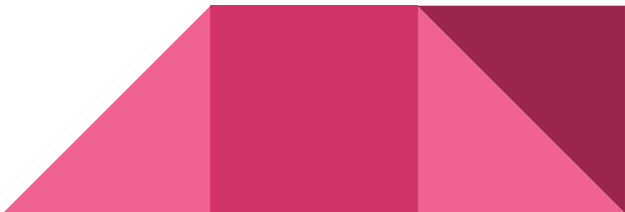
En Python se puede sugerir el tipo de cada parámetro que recibe una función, así como también el tipo del valor de retorno.

```
def saludar(nombre: str,  apellido: str = "González") -> None:
```

```
    print(f"Hola, {nombre} {apellido}!")
```

```
def calculo(num1: int,  num2: float = 1.0,  tup = (0, 0, 0)) -> float:
```

```
    return num1/num2 + tup[2]
```



Invocación de una función

```
def invocadora(param1, param2):
```

```
    # print(param1)
```

```
    print(param1(param2)**2)
```

```
def invocada(num: int = 0):
```

```
    return num%5
```

```
invocadora(invocada, 13)
```



Docstring

```
def alguna_funcion():
```

```
    """Esto es un docstring. Aquí va una explicación concisa de lo que hace la  
    función, los parámetros que recibe y/o los valores que devuelve  
    """
```

```
    # código
```

```
print(alguna_funcion.__doc__)
```



return, pass, continue

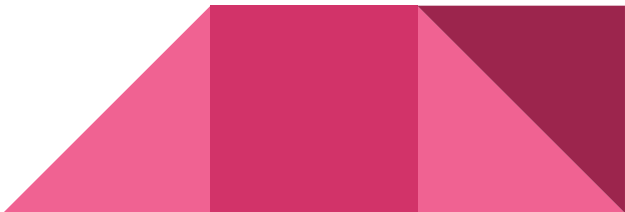
Una función termina de ejecutarse una vez que se realiza la sentencia **return**. De esta forma cualquier otro código que se encuentre luego de esta no se ejecutará.

```
def funcion(*args):  
    for i in args:  
        if type(i) is int:  
            return f'{i} es un entero'  
        else:  
            pass  
    print('fin de la función')
```

```
print(funcion)  
print(funcion())  
print(funcion('a', 'b', 3.4))  
print(funcion(9, 8, 7))
```

```
def funcion2(*args):  
    for i in args:  
        if i % 2 == 0:  
            continue  
        print(f'{i} es par')  
    print(i**2)
```

```
funcion2()  
funcion2(5, 7, 9)  
funcion2(4, 6, 8)  
funcion2(9, 8, 7)
```



Variables locales y globales

Las variables locales son las declaradas dentro de una función y las globales son declaradas en el programa principal.

```
x = 'principal'
```

```
def funcion1():  
    print('funcion1 ' + x)  
    #funcion2()
```

```
def funcion2():  
    #funcion1()  
    x = 'local'  
    print('funcion2 ' + x)
```

```
funcion2()
```



Variables locales y globales

Las variables locales son las declaradas dentro de una función y las globales son declaradas en el programa principal.

```
x = 1
a = 2

def funcion(a):
    x = 10
    a = 20

funcion(a)
print(x, a) # 1 2
```

```
x = 1
a = 2

def funcion(a):
    global x
    x = 10
    a = 20

funcion(a)
print(x, a) # 10 2
```



chr(), ord(), enumerate()

- **chr(ent)**: Recibe un entero y lo devuelve convertido a carácter según la tabla ascii.
- **ord(car)**: Recibe un carácter y lo devuelve convertido a entero según la tabla ascii.
- **enumerate(iter)**: Devuelve tuplas con el índice y el valor de cada elemento del iterable.



Ejercicio cifrado César

Se desea hacer un programa que implemente el cifrado César. La idea es codificar un mensaje para que no pueda ser leído hasta que sea descifrado. El cifrado consiste en desplazar alfabéticamente una cantidad de espacios determinados cada letra de un mensaje (mensaje: 'hola, pepe!' — codificado: 'ipmb, qfqf!').

Realizar una función para cifrar y otra para descifrar.

Hints: `string.ascii_lowercase`, `str.index()`, `ord()`, `chr()`, `for ind, val in enumerate(iter)`



Ejercicio 1

Escribir una función que reciba una muestra de números y devuelva el promedio.



Ejercicio 2

Escribe una función que reciba dos números y devuelva el resultado de dividir el primer número por el segundo número. Si el segundo número es cero, la función debe devolver "Error: no se puede dividir por cero".



Ejercicio 3

Escribe una función que reciba una cadena de texto y devuelva la misma cadena de texto pero sin espacios en blanco.



Ejercicio 4

Escribe una función que reciba una lista de cadenas de texto y devuelva una lista con las mismas cadenas de texto pero cada una debe estar invertida.

